Cyprus University of Technology

Τεχνολογικό Πανεπιστήμιο Κύπρου

Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Έγγραφο Υλοποίησης Πτυχιακής Εργασίας

Θέμα:

«CoachFlow: Διαδικτυακό Σύστημα Διαχείρισης Προπονητών και Πελατών»

Επιβλέπουσα Καθηγήτρια:
Ελίζα Αγγελίδου Λοΐζου

Φοιτητής:
Αντωνίου Ανδρέας
ΑΦΤ: 24892

Περιεχόμενα

0.1 Εισαγωγή . . . 3

0.1.1 Σκοπός . . . 3

0.1.2 Εμβέλεια . . . 3

0.1.3 Ορισμοί . . . 3

0.1.4 Επισκόπηση . . . 4

0.2 Περιγραφή Εργαλείων . . . 5

0.2.1 Frontend . . . 5

0.2.2 Backend . . . 5

0.2.3 Βάση Δεδομένων . . . 5

0.2.4 Εξωτερικές Υπηρεσίες . . . 5

0.3 Δυσκολίες που αντιμετωπίστηκαν . . . 7

0.4 Παραδείγματα κώδικα (Backend MVC) . . . 9

0.4.1 Εκκίνηση Εφαρμογής (bootstrap.php) . . . 9

0.4.2 Ρύθμιση CORS & Security Headers (cors.php) . . . 9

0.4.3 Εξουσιοδότηση Ρόλων (Auth.php) . . . 10

0.4.4 Σύνδεση με τη Βάση Δεδομένων (Database.php) . . . 11

0.4.5 Παράδειγμα Μοντέλου (UserModel.php) . . . 12

0.4.6 Παράδειγμα Controller (CoachController.php) . . . 13

0.4.7 Παράδειγμα Route (Login.php) . . . 14

0.4.8 Έκδοση Firebase Custom Token (GetFirebaseCustomToken.php) . . . 14

0.5 Παραδείγματα κώδικα (Frontend React) . . . 16

0.5.1 CSRF-Protected Fetch Wrapper (APIClient.js) . . . 16

0.5.2 Firebase Αρχικοποίηση & Custom Token Sign-In (firebase.js) . . . 17

0.5.3 Real-time Chat Component (ChatWindow.jsx) 18
0.6 Deployment Σχεδίαση 20
0.6.1 Τοπικό Περιβάλλον Ανάπτυξης (XAMPP) 20
0.6.2 Παραγωγικό Περιβάλλον (Hostinger) 20
0.6.3 Μεταβλητές Περιβάλλοντος (.env) 20
0.7 Περιπτώσεις Ελέγχου (Test Cases) 22

0.1 Εισαγωγή

0.1.1 Σκοπός

Ο σκοπός αυτού του εγγράφου υλοποίησης είναι να παρέχει ένα αναλυτικό εγχειρίδιο προγραμματισμού και να αναδείξει πρακτικά αποσπάσματα πραγματικού κώδικα σχετικά με την πλατφόρμα CoachFlow. Το έγγραφο τεκμηριώνει τεχνικές αποφάσεις, πραγματικά παραδείγματα κώδικα Backend (PHP MVC) και Frontend (React), δυσκολίες που ξεπεράστηκαν κατά τη διάρκεια της ανάπτυξης, και τον τρόπο εγκατάστασης και λειτουργίας του συστήματος.

0.1.2 Εμβέλεια

Η εμβέλεια αυτού του εγγράφου φτάνει μέχρι τη λεπτομερή επεξήγηση συγκεκριμένων τμημάτων κώδικα, όπως Controllers, Models, Routes, Frontend components και API integrations, των δυσκολιών που ξεπεράστηκαν, του deployment σε τοπικό και παραγωγικό περιβάλλον, και των βασικών περιπτώσεων ελέγχου.

0.1.3 Ορισμοί

API (Application Programming Interface)
Διασύνδεση προγραμματισμού εφαρμογών που επιτρέπει στο React Frontend να ανταλλάσσει δεδομένα με το PHP Backend με χρήση JSON.

Firebase Custom Tokens
Ειδικά διαμορφωμένα ασφαλή tokens που παράγονται δυναμικά από τον PHP Server χρησιμοποιώντας το Firebase Admin SDK (kreait/firebase-php). Επιστρέφονται στο React ώστε να πιστοποιηθεί η ταυτότητα του χρήστη και να αποκτήσει πρόσβαση στις υπηρεσίες Firebase (κυρίως Firestore για real-time messaging).

MVC (Model-View-Controller)
Αρχιτεκτονικό πρότυπο στο οποίο βασίζεται το PHP Backend. Τα Models διαχειρίζονται τα SQL queries, οι Controllers περιέχουν την επιχειρησιακή λογική, και τα Route αρχεία (.php endpoints) αποτελούν την είσοδο του REST API.

CSP (Content Security Policy)
Μηχανισμός ασφάλειας που περιορίζει τις πηγές από τις οποίες ο browser επιτρέπεται να φορτώσει πόρους, αποτρέποντας επιθέσεις τύπου XSS (Cross-Site Scripting).

CSRF (Cross-Site Request Forgery)
Τύπος επίθεσης όπου ένας κακόβουλος ιστότοπος εξαπατά τον browser να στείλει αίτημα σε άλλον ιστότοπο για λογαριασμό του συνδεδεμένου χρήστη. Αντιμετωπίζεται με synchronised token pattern (XSRF-TOKEN).

0.1.4 Επισκόπηση

- Στην Ενότητα 2 παρουσιάζεται το πλήρες tech stack με πραγματικές εκδόσεις πακέτων.
- Στην Ενότητα 3 αναφέρονται οι δυσκολίες που αντιμετωπίστηκαν και οι λύσεις που δόθηκαν.
- Στην Ενότητα 4 δίνονται αντιπροσωπευτικά παραδείγματα κώδικα Backend (PHP MVC).
- Στην Ενότητα 5 δίνονται αντιπροσωπευτικά παραδείγματα κώδικα Frontend (React).
- Στην Ενότητα 6 περιγράφεται η διαδικασία Deployment σε τοπικό και παραγωγικό περιβάλλον.
- Στην Ενότητα 7 αναλύονται βασικές περιπτώσεις ελέγχου (Test Cases).

0.2 Περιγραφή Εργαλείων

Για την υλοποίηση του CoachFlow επιλέχθηκε ένας πλήρως διαχωρισμένος (Decoupled) Tech Stack:

0.2.1 Frontend

Υλοποιήθηκε αποκλειστικά με React.js 19.1.1, χτισμένο μέσω του Vite 7.1.2 για ταχύτατο Development και Build. Η οπτική διεπαφή αναπτύχθηκε κυρίως με Material-UI (MUI) 6.1.6 και Bootstrap 5.3.8, ενώ η πλοήγηση διαχειρίζεται μέσω React Router DOM 7.9.1. Τα γραφήματα Analytics υλοποιήθηκαν με Recharts 3.7.0, τα Data Grids του Admin panel με MUI X Data Grid 7.22.2, και η εξαγωγή PDF με jsPDF 2.5.1 και jspdf-autotable 3.8.0. Τα stylesheets αναπτύχθηκαν σε SCSS μέσω Sass 1.92.1. Για μοναδικά αναγνωριστικά (π.χ. Firestore συνομιλίες) χρησιμοποιήθηκε η βιβλιοθήκη uuid 9.0.1.

0.2.2 Backend

Ο PHP Server κατασκευάστηκε σε αρχιτεκτονική MVC (Model-View-Controller), με **146 ξεχωριστά Route αρχεία** που αντιστοιχούν σε REST API endpoints. Χρησιμοποιήθηκε Composer για τη διαχείριση των εξής πακέτων:

- phpmailer/phpmailer 6.9: Αποστολή transactional emails μέσω Gmail SMTP.
- vlucas/phpdotenv 5.6: Φόρτωση μεταβλητών περιβάλλοντος από .env αρχείο.
- kreait/firebase-php 7.24: Firebase Admin SDK για έκδοση Custom Tokens (μηνύματα Firestore).
- stripe/stripe-php 19.1: Ενσωμάτωση Stripe API για PaymentIntents και επεξεργασία Webhooks.

0.2.3 Βάση Δεδομένων

Επιλέχθηκε η MySQL λόγω της σχεσιακής αξιοπιστίας της σε μοντέλα όπου ο Trainer διαχειρίζεται πολλούς Clients μέσω Foreign Keys. Η βάση δεδομένων ονομάζεται trainers-all-in-one και περιέχει πίνακες για χρήστες, σχέσεις προπόνησης, προγράμματα (marketplace & assignments), αγορές, καταγραφές διατροφής/προπόνησης, πρόοδο, analytics και authentication. Η πρόσβαση γίνεται μέσω MySQLi με prepared statements για αποτροπή SQL Injection.

0.2.4 Εξωτερικές Υπηρεσίες

- **Firebase 11.0.0:** Firestore για real-time messaging, Firebase Storage για αποθήκευση αρχείων, και Firebase Auth με custom tokens που εκδίδονται από τον PHP Server.

- **Stripe:** Stripe Elements (@stripe/react-stripe-js 5.4.1) για ασφαλή εισαγωγή καρτών, και PaymentIntents με Webhooks στο backend για ασύγχρονη επιβεβαίωση πληρωμών.
- **USDA FoodData Central API:** Αναζήτηση τροφών και θρεπτικών στοιχείων για το Nutrition module, υλοποιημένο μέσω USDAService.php με τοπική αποθήκευση αποτελεσμάτων (caching) στον πίνακα usda_foods.
- **Mailchimp API:** Διαχείριση email marketing (δημιουργία/αποστολή καμπανιών, audiences) από τον Admin dashboard μέσω MailchimpService.php.
- **Gmail SMTP / PHPMailer:** Transactional emails (επιβεβαίωση email, reset password, ειδοποιήσεις) μέσω MailService.php.

0.3 Δυσκολίες που αντιμετωπίστηκαν

Κατά τη διάρκεια της υλοποίησης παρουσιάστηκαν σημαντικές τεχνικές προκλήσεις:

- **CORS & Security Headers:** Επειδή το React SPA (Vite, port 5173) και το PHP API (Apache/XAMPP, port 80) τρέχουν σε διαφορετικά origins, ο browser απέκλειε τα cookie-based API requests. Η λύση υλοποιήθηκε στο cors.php: ρυθμίστηκαν δυναμικά επιτρεπόμενα origins μέσω AppConfig (υποστήριξη πολλαπλών URLs από το .env), ενεργοποιήθηκε Access-Control-Allow-Credentials, ορίστηκε αναλυτική Content Security Policy (CSP), και προστέθηκαν επιπλέον κεφαλίδες ασφαλείας (X-Frame-Options: DENY, X-XSS-Protection, X-Content-Type-Options: nosniff, Referrer-Policy).

- **Υβριδικό Authentication (Sessions & Firebase):** Το σύστημα χρησιμοποιεί PHP sessions για την εξουσιοδότηση των business endpoints, αλλά το real-time chat απαιτεί σύνδεση στο Firebase. Για να γεφυρωθεί αυτό το κενό, υλοποιήθηκε το GetFirebaseCustomToken.php: ο PHP Server χρησιμοποιεί το kreait/firebase-php SDK με service account JSON για έκδοση custom token. Το React το χρησιμοποιεί για signInWithCustomToken() στο Firebase Auth, αποκτώντας πρόσβαση στο Firestore χωρίς δεύτερο σύστημα login.

- **Stripe Webhooks & PaymentIntents:** Η ενσωμάτωση αξιόπιστων πληρωμών απαιτούσε χειρισμό ασύγχρονων σημάτων από τη Stripe. Η τιμή αγοράς δεν εμπιστεύεται ποτέ το frontend payload, το backend διαβάζει την τιμή από τη βάση δεδομένων και δημιουργεί το PaymentIntent. Ο τελικός έλεγχος γίνεται μέσω Stripe Webhook (StripeWebhook.php) με signature verification, ώστε να αποφευχθεί χειραγώγηση της τιμής από τον client.

- **Ενσωμάτωση USDA API & Caching:** Το USDA FoodData Central API επιστρέφει μεγάλες ποσότητες δεδομένων θρεπτικής αξίας ανά τρόφιμο. Η υλοποίηση του USDAService.php επιλέγει μόνο τα απαραίτητα πεδία (θερμίδες, πρωτεΐνες, υδατάνθρακες, λίπη) και τα αποθηκεύει στον πίνακα usda_foods για αποφυγή επαναλαμβανόμενων API calls, μείωση latency και αποφυγή υπέρβασης ορίου κλήσεων (rate limit).

- **Σύνθετη Διαχείριση Προγραμμάτων (Marketplace vs Direct Assignment):** Το σύστημα υποστηρίζει δύο ξεχωριστές ροές απόκτησης προγράμματος, αγορά μέσω Stripe (πίνακας program_purchases) και άμεση ανάθεση από Trainer (πίνακας program_assignments). Η πολυπλοκότητα εντοπίστηκε στο γεγονός ότι ο Trainee πρέπει να βλέπει και τα δύο είδη στην ίδια βιβλιοθήκη (My Workouts), απαιτώντας συνδυαστικό SQL query (UNION/JOIN) με ξεχωριστούς κανόνες ορατότητας (is_hidden flag για αγορές, πάντα ορατό για αναθέσεις).

- **Real-time Chat & Firestore Security Rules:** Η υλοποίηση του chat μέσω Firestore απαιτούσε σωστό σχεδιασμό security rules ώστε ένας χρήστης να διαβάζει και να γράφει μόνο στις δικές του συνομιλίες. Η λύση συνδύασε custom token claims (ρόλος χρήστη στο JWT) με κανόνες Firestore που ελέγχουν ότι το request.auth.uid ανήκει στους participants της εκάστοτε συνομιλίας.

- **Σύνδεση Trainer-Trainee & Κοινή Διαχείριση Πλάνων/Γευμάτων:** Μια από τις πιο σύνθετες προκλήσεις ήταν η υλοποίηση της σχέσης Trainer-Trainee με ταυτόχρονη υποστήριξη δύο ανεξάρτητων ρόλων στα ίδια δεδομένα. Ο Trainer πρέπει να μπορεί να

αναθέτει προγράμματα προπόνησης (workout plans) και πλάνα διατροφής (meal plans) απευθείας στον Trainee μέσω της σχέσης coaching_relationship. Παράλληλα, ο Trainee διατηρεί τη δυνατότητα να δημιουργεί δικά του προσωπικά προγράμματα και να βλέπει ταυτόχρονα τόσο τα δικά του όσο και αυτά που του έχει αναθέσει ο Trainer στην ίδια οθόνη. Αυτό απαιτούσε ξεχωριστούς πίνακες (user_workout_plans για προσωπικά, program_assignments για αναθέσεις Trainer) και συνδυαστικά SQL queries που ενοποιούν τα αποτελέσματα με σωστό διαχωρισμό προέλευσης, ώστε ο Trainee να γνωρίζει ποιο πρόγραμμα είναι δικό του και ποιο του αντιστοιχεί από τον Trainer.

- **Πρόσβαση Trainer στα Analytics του Trainee:** Ένα επιπλέον επίπεδο πολυπλοκότητας εντοπίστηκε στο γεγονός ότι ο Trainer χρειάζεται πρόσβαση στα analytics και το ιστορικό προόδου των Trainees του (βάρος, μετρήσεις σώματος, training volume, ιστορικό προπονήσεων), ενώ ο κάθε Trainee πρέπει να βλέπει μόνο τα δικά του δεδομένα. Η λύση βασίστηκε σε role-aware endpoints (π.χ. GetClientAnalytics.php) που επαληθεύουν πρώτα ότι υπάρχει ενεργή σχέση συνεργασίας (coaching_relationships.status = active) μεταξύ του Trainer και του ζητούμενου Trainee, πριν επιστρέψουν τα δεδομένα από τους πίνακες body_measurements, workout_sessions και weekly_volume_summary. Έτσι αποφεύγεται οποιαδήποτε μη εξουσιοδοτημένη πρόσβαση σε δεδομένα τρίτων.

0.4 Παραδείγματα κώδικα (Backend MVC)

0.4.1 Εκκίνηση Εφαρμογής (bootstrap.php)

Το bootstrap.php φορτώνεται σε κάθε request και είναι υπεύθυνο για τρεις βασικές λειτουργίες: (α) φόρτωση environment variables μέσω phpdotenv, (β) εκκίνηση της PHP session με HttpOnly cookie διάρκειας 24 ωρών και SameSite=Lax για αποτροπή CSRF, και (γ) δημιουργία και αποστολή CSRF token ως cookie XSRF-TOKEN (αναγνώσιμο από JavaScript).

```php
<?php
// load all the packages we installed with composer
require_once __DIR__ . '/../vendor/autoload.php';

// read the .env file so we can use our secret keys
$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/..');
$dotenv->load();

// only start the session if it isnt already running
if (session_status() !== PHP_SESSION_ACTIVE) {
    // keep the session alive for a whole day (86400 seconds)
    ini_set('session.cookie_lifetime', 86400);
    ini_set('session.gc_maxlifetime', 86400);
    session_set_cookie_params([
        'lifetime' => 86400,
        'path'     => '/',
        'httponly' => true,   // javascript cant see our cookie for xss attacks
        'samesite' => 'Lax'   // stops other websites from using our cookies for csrf attacks
    ]);
    session_start();
}

// if we dont have a csrf token yet, make a random one
if (empty($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}

// put the csrf token in a cookie so the frontend can read it
setcookie('XSRF-TOKEN', $_SESSION['csrf'], [
    'expires'  => 0,        // cookie dies when browser closes
    'path'     => '/',
    'httponly' => false,    // has to be false so javascript can actually read it
    'samesite' => 'Lax'
]);
```

0.4.2 Ρύθμιση CORS & Security Headers (cors.php)

Το cors.php ρυθμίζει τα επιτρεπόμενα origins δυναμικά μέσω AppConfig, ενεργοποιεί τα credentials για αποστολή session cookie, ορίζει αναλυτική Content Security Policy (CSP), και επιστρέφει HTTP 200 για OPTIONS preflight requests του browser.

```php
<?php
// start the session and load env variables
require_once(__DIR__ . '/../bootstrap.php');
// get our app settings like allowed URLs
require_once(__DIR__ . '/AppConfig.php');

// get the list of websites that are allowed to talk to us
$allowedOrigins = AppConfig::getAllowedOrigins();
// see where the request is coming from
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';

// if the request comes from a website we trust, let it through
if (in_array($origin, $allowedOrigins)) {
    header("Access-Control-Allow-Origin: " . $origin);
} else {
    // otherwise just allow the first one on our list as fallback
    header("Access-Control-Allow-Origin: " . $allowedOrigins[0]);
}

// tell the browser which request types we accept
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
// tell the browser which headers it can send us
header("Access-Control-Allow-Headers: Content-Type, Authorization, X-CSRF-
    Token");
// this is needed so cookies get sent along with the request
header("Access-Control-Allow-Credentials: true");
// we always respond with json
header("Content-Type: application/json; charset=UTF-8");

// dont let other sites put us inside an iframe (clickjacking protection)
header("X-Frame-Options: DENY");
// turn on the browsers built-in xss filter
header("X-XSS-Protection: 1; mode=block");
// stop browsers from guessing the file type
header("X-Content-Type-Options: nosniff");
header("Referrer-Policy: strict-origin-when-cross-origin");

// browsers send an OPTIONS request first to check if its safe, just say
    yes and stop
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header("HTTP/1.1 200 OK");
    exit();
}
```

0.4.3 Εξουσιοδότηση Ρόλων (Auth.php)

Η συνάρτηση checkAuth() ελέγχει αν υπάρχει ενεργή PHP session (logged_in === true) και αν ο ρόλος του χρήστη (user_privileges) ανήκει στους επιτρεπόμενους ρόλους που δηλώνει ο κάθε endpoint. Σε αποτυχία επιστρέφει HTTP 401 (μη συνδεδεμένος) ή HTTP 403 (ανεπαρκή δικαιώματα).

```php
<?php
// this function runs at the top of every protected page
// you pass it a list of roles that are allowed, like ['trainer', 'admin']
function checkAuth(array $allowedRoles)
```

```php
{
    // if the user is not logged in at all, kick them out with a 401
    if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
        http_response_code(401);
        echo json_encode(['message' => 'You are not authorized to access this resource.']);
        exit();
    }
    // if they are logged in but their role is not on the allowed list, also kick them out
    // a trainee trying to access a trainer page gets a 403 here
    if (!in_array($_SESSION['user_privileges'], $allowedRoles)) {
        http_response_code(403);
        echo json_encode(['message' => 'You are not authorized to access this resource.']);
        exit();
    }
    // if we get here the user is fine, just continue normally
}
```

0.4.4 Σύνδεση με τη Βάση Δεδομένων (Database.php)

Η κλάση δημιουργεί το αντικείμενο mysqli και συνδέεται στον MySQL server, διαβάζοντας παραμέτρους από Environment Variables (με fallback τιμές για τοπική ανάπτυξη). Χρησιμοποιεί lazy initialization: η σύνδεση δημιουργείται μόνο στην πρώτη κλήση της connect().

```php
<?php
class Database {
    // all the info we need to connect to mysql
    private $host, $dbName, $username, $password, $port, $socket;
    // we store the connection here so we dont open it more than once
    private $conn;

    public function __construct() {
        // read from the .env file, if something is missing use a default value
        $this->host     = $_ENV['DB_HOST'] ?? 'localhost';
        $this->dbName   = $_ENV['DB_NAME'] ?? 'trainers-all-in-one';
        $this->username = $_ENV['DB_USER'] ?? 'root';
        $this->password = $_ENV['DB_PASS'] ?? '';
        $this->port     = $_ENV['DB_PORT'] ?? 3306;
        // on mac with xampp the socket file lives here
        $this->socket   = $_ENV['DB_SOCKET']
                          ?? '/Applications/XAMPP/xamppfiles/var/mysql/mysql.sock';
    }

    public function connect() {
        // only connect if we havent connected yet, no point opening it twice
        if ($this->conn === null) {
            // make mysqli throw exceptions instead of silent failures
            mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
            try {
```

```
                $this->conn = new \mysqli(
                    $this->host, $this->username, $this->password,
                    '', $this->port, $this->socket
                );
                // pick the right database
                if (!$this->conn->select_db($this->dbName)) {
                    throw new \Exception("Database does not exist");
                }
                // use utf8mb4 so emojis and greek characters dont break
                $this->conn->set_charset('utf8mb4');
            } catch (\Exception $e) { /* something went wrong, handle it */
    }
        }
        // give back the connection so models can use it
        return $this->conn;
    }
}
```

0.4.5 Παράδειγμα Μοντέλου (UserModel.php)

Τα Models αναλαμβάνουν τις εντολές SQL χρησιμοποιώντας prepared statements για αποτροπή SQL Injection. Το παρακάτω απόσπασμα δείχνει την αποθήκευση νέου χρήστη με bind_param για ασφαλή παρεμβολή παραμέτρων.

```
<?php
class UserModel {
    public $conn;
    // the actual table name in the database
    private $table = 'user';
    // these match the columns in the user table
    public $UserID, $username, $email, $password;
    public $UserPrivileges, $IsVerified;

    public function __construct() {
        // open the database connection as soon as we create a UserModel
        $this->conn = (new Database())->connect();
    }

    public function saveUser() {
        // the question marks are placeholders, we fill them in safely
    below
        // this way hackers cant do sql injection by typing weird stuff in
    the form
        $query = "INSERT INTO {$this->table}
                  (userid, email, password, username, role, isverified)
                  VALUES (?, ?, ?, ?, ?, ?)";
        $stmt = $this->conn->prepare($query);
        // if prepare failed something is very wrong, just stop
        if (!$stmt) { return false; }

        // plug in the real values, all strings so we use "ssssss"
        $stmt->bind_param("ssssss",
            $this->UserID,
            $this->email,
            $this->password,   // this is already hashed with password_hash
    (), not plain text
```

```
            $this->username,
            $this->UserPrivileges,
            $this->IsVerified
        );

        // run it and return true if it worked
        if ($stmt->execute()) { $stmt->close(); return true; }
        // if it failed write the error to the log so we can check later
        error_log("Execute failed: " . $stmt->error);
        $stmt->close();
        return false;
    }
}
```

0.4.6 Παράδειγμα Controller (CoachController.php)

Ο Controller χειρίζεται τη λογική ομαδοποίησης δεδομένων. Στο παρακάτω απόσπασμα λαμβάνεται ο τρέχων προπονητής ενός Trainee, τα μηνύματά του, και η λίστα διαθέσιμων προπονητών, αξιοποιώντας έτοιμες κλήσεις του CoachModel και επιστρέφοντας ένα δομημένο JSON αντικείμενο.

```
<?php
// bring in the model that talks to the database
require_once '../models/CoachModel.php';
// bring in the auth checker
require_once '../config/Auth.php';

class CoachController {
    // we keep one CoachModel and reuse it
    private $coachModel;

    public function __construct() {
        $this->coachModel = new CoachModel();
    }

    // builds up all the coach data we need for the frontend in one go
    public function getCoachData($userId) {
        try {
            // cant do anything without knowing who the user is
            if (!$userId) throw new Exception('User ID is required');

            // start with an empty response, we fill it in below
            $response = [
                'success'          => true,
                'currentCoach'     => null,
                'messages'         => [],
                'availableCoaches' => []
            ];

            // check if this user already has a coach assigned
            $currentCoach = $this->coachModel->getCurrentCoach($userId);
            if ($currentCoach) {
                // they have a coach, put it in the response
                $response['currentCoach'] = $currentCoach;
                // also grab their conversation with that coach
```

```
                $response['messages'] = $this->coachModel->getMessages(
                    $userId, $currentCoach['id']
                );
            }
            // always show the full list of available coaches too
            $response['availableCoaches'] =
                $this->coachModel->getAvailableCoaches();
            return $response;

        } catch (Exception $e) {
            // something broke, tell the frontend what happened
            return ['success' => false, 'message' => $e->getMessage()];
        }
    }
}
```

0.4.7 Παράδειγμα Route (Login.php)

Κάθε API endpoint αντιστοιχεί σε ένα ξεχωριστό .php αρχείο στον φάκελο routes/. Το Login.php δέχεται POST αίτημα, φορτώνει το cors.php (που αρχικοποιεί session και CSRF), αποκωδικοποιεί τα δεδομένα JSON, δημιουργεί τον UserController, και επιστρέφει JSON απάντηση.

```
<?php
// this sets up cors, starts the session and generates the csrf token
require_once __DIR__ . "/../config/cors.php";
// bring in the controller that handles login logic
include_once("../controllers/UserController.php");

// only run if the request is a POST, ignore everything else
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // read the json the frontend sent us and turn it into a php array
    $inputData = json_decode(file_get_contents("php://input"), true);
    $userController = new UserController();
    // pass the data to the controller and get back the result
    $response = $userController->login($inputData);
    // send the result back to the frontend as json
    echo json_encode($response);
}
```

0.4.8 Έκδοση Firebase Custom Token (GetFirebaseCustomToken.php)

Το endpoint εκδίδει Firebase Custom Token για τον τρέχοντα session χρήστη, χρησιμοποιώντας το kreait/firebase-php SDK με service account JSON. Ελέγχεται πρώτα η εξουσιοδότηση μέσω checkAuth, και το token εκδίδεται με user_id ως UID και ρόλο χρήστη ως custom claim.

```
<?php
// setup cors and session first
include_once("../config/cors.php");
include_once("../config/Auth.php");
// only logged in users can get a firebase token, guests cant
```

```php
checkAuth(['trainee', 'trainer', 'admin']);

// load composer packages including the firebase sdk
require_once __DIR__ . '/../../vendor/autoload.php';

// grab the user id from the session, firebase needs it as a string
$uid = (string)$_SESSION['user_id'];
// this is the secret file that proves we are allowed to use firebase
$serviceAccountPath = __DIR__ . '/../config/firebase-service-account.json';

use Kreait\Firebase\Factory;

try {
    // connect to firebase using our service account
    $factory = (new Factory())->withServiceAccount($serviceAccountPath);
    $auth = $factory->createAuth();

    // add the user role as extra info inside the token
    // firestore security rules can read this later
    $claims = [];
    if (isset($_SESSION['privileges'])) {
        $claims['role'] = $_SESSION['privileges'];
    }

    // make the token for this specific user
    $customToken = $auth->createCustomToken($uid, $claims);
    // send it back to the frontend
    echo json_encode([
        'success' => true,
        'token'   => $customToken->toString(),
        'claims'  => $claims,
    ]);
} catch (Throwable $e) {
    // something went wrong with firebase, tell the frontend
    http_response_code(500);
    echo json_encode(['success' => false, 'error' => $e->getMessage()]);
}
```

0.5 Παραδείγματα κώδικα (Frontend React)

0.5.1 CSRF-Protected Fetch Wrapper (APIClient.js)

Η κλάση APIClient αποτελεί τον κεντρικό μηχανισμό επικοινωνίας του Frontend με το REST API. Διαβάζει αυτόματα το XSRF-TOKEN cookie (που έχει αποσταλεί από το bootstrap.php) και το προσθέτει ως header X-CSRF-Token σε κάθε state-changing request (POST/PUT/DELETE). Περνά πάντα credentials: 'include' για την αποστολή του session cookie σε cross-origin requests.

```
class APIClient {
  // every request goes through here so we never forget the csrf token
  static async fetch(url, options = {}) {
    // grab the csrf token the backend put in our cookies
    const csrfToken = this.getCookie('XSRF-TOKEN');

    // GET and HEAD requests dont change anything so they dont need the
   token
    // POST, PUT, DELETE do change stuff so they need it
    if (options.method && !['GET', 'HEAD'].includes(options.method.
   toUpperCase())) {
      options.headers = {
        ...options.headers,
        'X-CSRF-Token': csrfToken || ''
      };
    }

    // always send the session cookie so the backend knows who we are
    options.credentials = 'include';
    return fetch(url, options);
  }

  // helper to find a cookie by name from the browser
  static getCookie(name) {
    return document.cookie
      .split('; ')
      .find(row => row.startsWith(name + '='))
      ?.split('=')[1];
  }

  // shortcut for sending a POST request with json data
  static async post(url, data, options = {}) {
    const response = await this.fetch(url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', ...options.headers },
      // turn the javascript object into a json string
      body: JSON.stringify(data),
      ...options
    });
    // if the server returned an error code throw so the caller can handle
   it
    if (!response.ok)
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    // parse the json the server sent back and return it
    return response.json();
```

```
  }
}

export default APIClient;
```

0.5.2 Firebase Αρχικοποίηση & Custom Token Sign-In (firebase.js)

Το firebase.js αρχικοποιεί την εφαρμογή Firebase με environment variables του Vite, εξάγει τα Firestore, Storage και Auth αντικείμενα, και παρέχει τη συνάρτηση signInWithBackendSession() που λαμβάνει custom token από το backend endpoint και κάνει signInWithCustomToken() στο Firebase Auth.

```
// firebase sdk imports
import { initializeApp, getApps } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';
import { getAuth, signInWithCustomToken } from 'firebase/auth';

// all the keys come from the .env file, never hardcode them here
const firebaseConfig = {
  apiKey:            import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain:        import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId:         import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket:     import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId:             import.meta.env.VITE_FIREBASE_APP_ID,
};

// dont create a second firebase app if one already exists (hot reload
    problem)
const app = getApps().length ? getApps()[0] : initializeApp(firebaseConfig)
    ;

// export these so any file in the project can import and use them
export const db      = getFirestore(app); // the chat database
export const storage = getStorage(app);   // for uploaded files
export const auth    = getAuth(app);      // for logging into firebase

// this is how we sign into firebase without a second login screen
// we ask our own backend for a token and use that to get into firebase
export async function signInWithBackendSession() {
  const endpoint = import.meta.env.VITE_API_BASE_URL
    + '/backend/src/routes/GetFirebaseCustomToken.php';
  try {
    // ask the backend for the token, send cookies so it knows who we are
    const res  = await fetch(endpoint, { credentials: 'include' });
    const data = await res.json();
    // if the backend said no, return an error and dont crash
    if (!res.ok || !data.success)
      return { user: null, error: data?.error || 'token_fetch_failed' };
    // use the token to sign into firebase
    const userCred = await signInWithCustomToken(auth, data.token);
    // return the firebase user so the rest of the app can use it
    return { user: userCred.user, claims: data.claims, error: null };
  } catch (e) {
```

```
    // network problems or firebase being down
    return { user: null, error: 'network_error', detail: String(e) };
  }
}
```

0.5.3 Real-time Chat Component (ChatWindow.jsx)

Το ChatWindow είναι ένα React component που λαμβάνει δεδομένα (messages, firebaseUser, conversations) από το ChatProvider context. Χρησιμοποιεί useEffect για αυτόματη κύλιση στο τελευταίο μήνυμα και σήμανση μηνυμάτων ως αναγνωσμένα, και useMemo για αποτελεσματική αναγνώριση του άλλου συμμετέχοντα χωρίς περιττούς επανυπολογισμούς.

```
import React, { useRef, useEffect, useMemo } from 'react';
// get all the chat state from the context provider
import { useChat } from '../../context/ChatProvider';

export function ChatWindow() {
  // pull out everything we need from the chat context
  const {
    messages, firebaseUser, activeConversationId,
    conversations, allUsers, markAsRead
  } = useChat();
  // this ref points to an invisible div at the bottom so we can scroll to
   it
  const bottomRef = useRef(null);

  // find the full conversation object for the one the user has open
  // useMemo so we dont recalculate this every single render
  const activeConversation = useMemo(
    () => conversations.find(c => c.id === activeConversationId),
    [conversations, activeConversationId]
  );

  // figure out who the other person in this chat is (not us)
  const otherParticipantUid = useMemo(() => {
    const participants = activeConversation?.participants || [];
    // find the uid that isnt ours
    return participants.find(p => String(p) !== String(firebaseUser?.uid));
  }, [activeConversation, firebaseUser]);

  // turn the other persons uid into a readable name
  const otherDisplayName = useMemo(() => {
    if (!otherParticipantUid) return 'Conversation';
    const match = allUsers.find(u =>
      String(u.userId || u.id) === String(otherParticipantUid)
    );
    // try different field names because different endpoints return
   different shapes
    return match?.displayName || match?.username || otherParticipantUid;
  }, [otherParticipantUid, allUsers]);

  // whenever new messages come in, mark the ones we havent read yet
  useEffect(() => {
    if (messages.length > 0 && firebaseUser) {
      messages.forEach(msg => {
```

```
        if (!msg.readBy?.includes(firebaseUser.uid)) markAsRead?.(msg.id);
      });
    }
  }, [messages, firebaseUser, markAsRead]);

  // every time messages change, scroll down so the latest one is visible
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  // if no conversation is selected just show a placeholder
  if (!activeConversationId)
    return <div>Select or start a conversation.</div>;

  return (
    <div className="d-flex flex-column h-100">
      {/* name of the person we are talking to */}
      <div className="chat-header">{otherDisplayName}</div>
      <div className="chat-messages">
        {messages.map(msg => (
          // our messages go on the right, theirs go on the left
          <div key={msg.id} className={
            String(msg.senderId) === String(firebaseUser?.uid)
              ? 'msg-own' : 'msg-other'
          }>
            {msg.text}
          </div>
        ))}
        {/* invisible div we scroll to so the latest message is always in view */}
        <div ref={bottomRef} />
      </div>
    </div>
  );
}
```

0.6 Deployment Σχεδίαση

0.6.1 Τοπικό Περιβάλλον Ανάπτυξης (XAMPP)

Για την τοπική ανάπτυξη χρησιμοποιήθηκε το XAMPP (Apache + MySQL + PHP 8.x) σε macOS. Η δομή εγκατάστασης ήταν:

- Το backend τοποθετήθηκε στον Apache Document Root (htdocs/CoachFlow/backend/) και εξυπηρετείται στο http://localhost/CoachFlow.
- Ο MySQL socket ρυθμίστηκε μέσω DB_SOCKET στο .env (/Applications/XAMPP/xamppfiles/va
- Το frontend εκτελείται ανεξάρτητα μέσω npm run dev (Vite, port 5173).
- Το VITE_API_BASE_URL ρυθμίστηκε σε http://localhost/CoachFlow.
- Τα Composer dependencies εγκαταστάθηκαν με composer install στον φάκελο backend/.
- Τα npm dependencies εγκαταστάθηκαν με npm install στον φάκελο frontend/.

0.6.2 Παραγωγικό Περιβάλλον (Hostinger)

Για το production deployment χρησιμοποιείται Hostinger με PHP 8.x και MySQL:

- Το React SPA γίνεται build με npm run build, παράγοντας στατικά αρχεία στον frontend/dist/ φάκελο, τα οποία ανεβαίνουν στον web server.
- Το PHP backend ανεβαίνει μαζί με τα vendor/ Composer dependencies.
- Τα .env αρχεία ΔΕΝ ανεβαίνουν (.gitignore) και ρυθμίζονται απευθείας μέσω hosting panel.
- Το HTTPS είναι υποχρεωτικό για το Stripe API και τα secure cookies.
- Ο Stripe Webhook (StripeWebhook.php) καταχωρείται στο Stripe Dashboard με webhook signing secret για signature verification.
- Τα session cookie params τίθενται σε secure: true σε production (APP_ENV=production).

0.6.3 Μεταβλητές Περιβάλλοντος (.env)

Όλα τα κλειδιά και credentials αποθηκεύονται σε environment variables και δεν εκτίθενται ποτέ στο frontend bundle. Οι βασικές backend μεταβλητές είναι:

- DB_HOST, DB_NAME, DB_USER, DB_PASS, DB_PORT, DB_SOCKET: Παράμετροι σύνδεσης MySQL.

- STRIPE_SECRET_KEY: Για server-side PaymentIntents και επαλήθευση webhooks.
- MAILCHIMP_API_KEY, MAILCHIMP_DEFAULT_AUDIENCE_ID: Για το email marketing dashboard.
- USDA_API_KEY: Για κλήσεις στο FoodData Central API.
- FRONTEND_URL: Επιτρεπόμενα origins για CORS (πολλαπλά URLs διαχωρισμένα με κόμμα).
- APP_ENV: Καθορίζει αν το σύστημα τρέχει σε development ή production λειτουργία.

Τα αντίστοιχα frontend environment variables (VITE_*) ρυθμίζουν το API base URL, τα Firebase credentials και το Stripe publishable key.

0.7 Περιπτώσεις Ελέγχου (Test Cases)

Για την επιβεβαίωση της αξιόπιστης λειτουργίας (QA/Integration Testing) αξιολογήθηκαν συστηματικά τα εξής:

- **TC01 --- Εγγραφή & Επιβεβαίωση Email:** Νέος χρήστης εγγράφεται με username/email/password και ρόλο (trainee/trainer). Το SignUp.php αποθηκεύει τον χρήστη με isverified=0 και αποστέλλει verification email μέσω PHPMailer/Gmail SMTP. Ο χρήστης κάνει κλικ στο link επιβεβαίωσης και ο λογαριασμός ενεργοποιείται (isverified=1). **Αποτέλεσμα:** Επιτυχές.
- **TC02 --- Σύνδεση & Δημιουργία Session:** Εγγεγραμμένος και επιβεβαιωμένος χρήστης κάνει login μέσω Login.php. Ελέγχθηκε ότι δημιουργείται PHP Session με logged_in=true, user_id και user_privileges, και ότι αποστέλλεται το XSRF-TOKEN cookie αναγνώσιμο από το JavaScript. **Αποτέλεσμα:** Επιτυχές.
- **TC03 --- Προστασία API (Unauthorized Access):** Κλήση προστατευμένου endpoint (π.χ. GetTrainerClients.php) χωρίς ενεργό session ή με λάθος role (π.χ. trainee αντί trainer). Αναμένεται HTTP 401/403. **Αποτέλεσμα:** Επιτυχές --- Το Auth.php απέρριψε την κλήση ελέγχοντας τα privileges.
- **TC04 --- Αγορά Προγράμματος μέσω Stripe:** Trainee επιλέγει published πρόγραμμα και εισάγει Stripe test card (4242 4242 4242 4242). Ελέγχθηκε η δημιουργία PaymentIntent στο backend με τιμή από τη βάση, η σωστή εμφάνιση Stripe Elements, η καταγραφή αγοράς ως pending, και η μετάβαση σε completed μέσω Webhook (signature verified). **Αποτέλεσμα:** Επιτυχές.
- **TC05 --- Ανάθεση Προγράμματος (Direct Assignment):** Trainer επιλέγει Trainee από το My Clients και αναθέτει υπάρχον πρόγραμμα. Ελέγχθηκε ότι καταγράφεται ορθά στον πίνακα program_assignments με τις σωστές συνδέσεις Foreign Keys, και ότι ο Trainee βλέπει το πρόγραμμα αμέσως στο My Workouts. **Αποτέλεσμα:** Επιτυχές.
- **TC06 --- CSRF Προστασία:** Αποστολή POST request σε state-changing endpoint χωρίς X-CSRF-Token header ή με λανθασμένο token. Αναμένεται απόρριψη του αιτήματος. **Αποτέλεσμα:** Επιτυχές. Το verify_csrf.php απέρριψε το αίτημα ελέγχοντας ότι το header token ταιριάζει με το session token.
- **TC07 --- Real-time Chat (Firebase Firestore):** Trainee και Trainer ανοίγουν ταυτόχρονα το Messages tab. Ελέγχθηκε η έκδοση Custom Token από το backend, η επιτυχής σύνδεση μέσω signInWithCustomToken(), η real-time εμφάνιση μηνυμάτων και στις δύο πλευρές, και η σωστή σήμανση μηνυμάτων ως αναγνωσμένων. **Αποτέλεσμα:** Επιτυχές.
- **TC08 --- Αναζήτηση USDA & Καταγραφή Διατροφής:** Trainee αναζητά τρόφιμο στο Nutrition module. Ελέγχθηκε ότι το USDAService.php καλεί το USDA FoodData Central API, επιστρέφει σωστά macros (θερμίδες, πρωτεΐνες, λίπη, υδατάνθρακες), αποθηκεύει στον πίνακα usda_foods για caching, και ότι η καταγραφή γεύματος αποθηκεύεται στον πίνακα food_logs. **Αποτέλεσμα:** Επιτυχές.

- **TC09 --- Διαχείριση Χρηστών (Admin):** Admin απενεργοποιεί λογαριασμό χρήστη μέσω UserDisable.php. Ελέγχθηκε ότι το endpoint απαιτεί admin role, ότι ο χρήστης αποθηκεύεται με disabled=1, και ότι ο απενεργοποιημένος χρήστης αδυνατεί να κάνει login. Επίσης ελέγχθηκε η επαναφορά (UserEnable.php) και το force reset password (ForceResetPassword.php). **Αποτέλεσμα:** Επιτυχές.

- **TC10 --- Reset Password (Email Flow):** Χρήστης ζητά επαναφορά κωδικού μέσω email. Ελέγχθηκε ότι αποστέλλεται email με μοναδικό token (αποθηκευμένο στον πίνακα resetpasstoken), ότι το token λήγει μετά από καθορισμένο χρόνο, και ότι ο νέος κωδικός αποθηκεύεται με password_hash() χωρίς να επιτρέπεται χρήση ληγμένου token. **Αποτέλεσμα:** Επιτυχές.